



COOL: continual online on-device learning for human activity recognition enhanced by KANs

Mengxi Liu^{2,1} · Sizhen Bian^{2,1} · Vitor Fortes Rey^{2,1} · Daniel Geißler² · Pixi Kang³ · Bo Zhou^{2,1} · Paul Lukowicz^{2,1}

Received: 14 October 2025 / Accepted: 17 January 2026
© The Author(s) 2026

Abstract

Variations in deployment parameters such as new users or sensor locations are a major concern in sensor-based human activity recognition (HAR), accounting for a major source of performance drop from in-lab experiments to in-the-wild deployment. While fine-tuning a pre-trained model using data acquired during deployment can reduce such performance loss, it often leads to catastrophic forgetting. The hardware constraints of edge devices also limit the options of continual learning techniques. To address these challenges, we introduce COOL, a continual online on-device learning method leveraging Kolmogorov-Arnold Networks (KANs). Our method exploits the inherent plasticity of KANs. COOL is evaluated through two HAR scenarios (utilizing bio-impedance and Inertial Measurement Unit (IMU) signals separately) to demonstrate its performance in addressing both the catastrophic forgetting issue and concept drift issue caused by new targets (users and sensor locations). A significant average overall performance improvement of around 6.82% and 9.80% in upper- and lower-body fitness activity recognition and 16.65% improvement in Locomotion activity recognition was observed from the experiment compared to the SotA methods. Specifically, the on-device trained model demonstrated the memory-retention ability with a forgetting measurement score of less than 0.01, 100-fold better than SotA on-device HAR continual learning methods. Finally, we evaluate the hardware performance of the proposed COOL model with the STM32H743 microcontroller, an on-device inference and training with a time cost of 44.30 ms and 46.03 ms is observed, guaranteeing real-time model adaptation.

Keywords Online on-device training · Continual learning · Human activity recognition · Kolmogorov–Arnold networks

1 Introduction

Human Activity Recognition (HAR) is an important research field with wide-ranging, promising potential applications in healthcare (Wang et al. 2019), sports (Host and Ivašić-Kos 2022), and industry (Nwakanma et al. 2021), which have been extensively studied in the pervasive computing community over decades. Continual learning is an important aspect for the real-life in-the-wild deployment of HAR,

where new variations like users and sensor locations often occur. On the theoretical side, continuous learning stands in the middle of two major challenges in HAR: concept drift and catastrophic forgetting. Concept drift is the unforeseeable changes in the underlying distribution of streaming data over time, resulting from the target variations of the users or the sensor location (Lu et al. 2018). Catastrophic forgetting is the phenomenon where artificial neural networks forget previously learned information upon continual learning of new tasks (Chen and Liu 2018). Putting into practice especially in the scope of pervasive computing, the operational modes of continual learning can be categorized as offline, online and on-device training. Offline training refers to training machine learning models with a fixed dataset before being deployed in the target scenarios which might involve new variants. Online training refers to adjusting the model as it receives new data, without access to the original training data during the offline phase (Yu et al. 2021). Both offline and online training do not specifically imply training

✉ Mengxi Liu
mliu@rptu.de

Sizhen Bian
firstname.lastname@dfki.de

¹ RPTU, Kaiserslautern, Germany

² German Research Center for Artificial Intelligence (DFKI), Kaiserslautern, Germany

³ Tsinghua University, Beijing, China

infrastructure or hardware requirements. On-device training is a mode with online training as basis, considering the limitations of hardware resources on edge devices, like limited memory and computation capability.

Among the state-of-the-art (SotA), as elaborated in Sect. 2, the challenges surrounding continual learning are often addressed by dataset expansion by including more subjects which can help address concept drift in HAR models by improving generalization and reducing overfitting (Ariza-Colpas et al. 2022).

However, collecting large amounts of data is costly and may not fully resolve concept drift, especially for personalized activities or rapid behavior changes (Vrigkas et al. 2015). On-field dynamic model updates, or online learning, offer a better solution by allowing models to adapt as new sensor data is received. However, on-device training with resource-constrained devices like Microcontroller Unit (MCUs) is challenging due to their limited memory and processing power. To address this, on-device partial fine-tuning techniques have been proposed, such as re-training only the classification layers, significantly reducing resource demand (Bian et al. 2024; Kang et al. 2024). However, these SotA methods focus on addressing concept drift with efficient methods allowing on-device continual learning, as catastrophic forgetting remains overlooked.

To address the above challenges, we propose COOL, a Continual online on-device Learning solution for HAR to tackle both concept drift and catastrophic forgetting at edge devices, by taking advantage of the local plasticity and global stability properties of Kolmogorov-Arnold networks (KANs).

Overall, in this work, we present the following contributions:

1. A continual online on-device learning method using Convolutional Neural Networks (CNN) as feature extractor and KANs as classifier was introduced to address both concept drift and catastrophic forgetting issues in HAR tasks.
2. A hardware-efficient training engine for fine-tuning KAN based classifier was proposed, enabling both inference and online training of the KAN-based classifier on STM32H743 microcontroller.
3. We extensively evaluated COOL through three HAR datasets including target variations of users and sensor locations. While COOL reaches similar concept-drift performance, it outperforms the SotA methods for on-device continual learning at alleviating catastrophic forgetting measurement by 100 fold.
4. The hardware performance including the latency and power efficiency was evaluated. The results show that it takes only around 46.3 ms and a dynamic power consumption of around 23 mW to perform the model fine-tuning on the STM32H743, which is feasible for real-time deployment.

2 Related work

This section introduces the recent related works from continual learning, online training, and the utilization of KANs in HAR, Table 1 summarizes and compares our approach with prior studies.

2.1 Continual learning in HAR

Continual learning can be categorized into three groups: class, task, and domain continual learning (Van de Ven et al. 2022), each with promising potential for addressing the challenge of deploying HARs in real-world environments. The first two types allow neural networks to recognize new activities while retaining the ability to recognize old ones, without requiring complete retraining. These approaches have been widely studied in various works (Lee et al. 2020;

Table 1 Comparison between related works and our work

Authors (year)	Type	Continual learning	On-device learning	Edge platform	Method	Classifier
Vakili and Rezaei (2021)	Class	Yes	No	No		MLP
Siirtola and Rönning (2021)	Class and domain	Yes	No	No	Auxiliary models	MLP
Leite and Xiao (2022)	Class and domain	Yes	No	No	Memory replay	MLP
Adaimi and Thomaz (2022)	Domain	Yes	No	No	Auxiliary models	MLP
Schiemer et al. (2023)	Class	Yes	No	No	Memory replay	MLP
Tang et al. (2024)	Class	Yes	No	No	Auxiliary models	MLP
Liu et al. (2024)	Class	Yes	No	No	Feature manipulation	KAN
Pau and Ambrose (2022)	Domain	No	Yes	ARM Cortex M7	Fine-tune	MLP
Craighero et al. (2023)	Domain	No	Yes	STM32L4	Fine-tune	CNN
Kang et al. (2024)	Domain	No	Yes	GAP9&STM32F7	Fine-tune	MLP
Bian et al. (2024)	Domain	No	Yes	GAP9	Fine-tune	MLP
Saha et al. (2024)	Domain	No	Yes	Smartphone	Fine-tune	MLP
Ours	Domain	Yes	Yes	STM32H743	Feature manipulation	KAN

Jin et al. 2021; Shanahan et al. 2021). Domain continual learning, on the other hand, refers to a scenario where a model learns to perform a task across different data distributions sequentially. In this case, the task remains the same across domains, but the input distribution changes, while the output space (e.g., set of classes) typically remains constant. Many solutions have been proposed to mitigate the problem of catastrophic forgetting in continual learning, such as Elastic Weight Consolidation (EWC), which uses parameter regularization (Liu et al. 2018), Deep Generative Replay (DGR), and auxiliary models that generate previous samples (Shin et al. 2017), or by using memory to store old samples. Consequently, many researchers have adapted continual learning to address the concept drift challenge in HAR tasks (Siirtola and Rönning 2021; Leite and Xiao 2022; Adaimi and Thomaz 2022), enabling models to continually learn to recognize new activities (Vakili and Rezaei 2021; Schiemer et al. 2023; Tang et al. 2024). For example, Schiemer et al. (2023) presents an end-to-end solution for online continual learning in HAR, addressing unique challenges such as streaming sensor data and sparse labeling. They performed outlier detection on small batches, stored the outlier samples in memory, and then used a semi-supervised learning method to process the in-memory buffered samples and discover new activities. The work in Siirtola and Rönning (2021) demonstrated an ensemble-based personalized HAR solution, where greater weight is given to ensemble base models most suited to the current context, allowing the system to adapt to different contexts and changes in context. However, many existing approaches often require extensive hardware resources, such as memory to store additional samples and computation power for multiple model inferences. This makes these methods difficult to deploy on edge devices with limited hardware resources. In this work, we integrated performing the continual learning schema on edge devices.

2.2 Online training on edge devices in HAR

While neural network inference on microcontrollers has been extensively studied in existing works (Bian and Lukowicz 2021; Ingolfsson et al. 2024, 2023), with the help of model compression techniques such as pruning and quantization (Liang et al. 2021; Nagel et al. 2021) and lightweight frameworks like TensorFlowLite (David et al. 2021), there has been limited research on online neural network training on MCUs in recent years. Despite the important advantages of online training on edge devices in HAR applications, such as privacy preservation, reduced data transmission latency, improved energy efficiency, personalization, and scalability (Schiemer et al. 2023), it remains challenging because online training requires significantly more memory

and computational resources due to the complexity of back-propagation. Additionally, current neural network inference frameworks do not support training on MCUs, leading to increased time consumption and difficulty when performing online training. Recently, several works have attempted to address the concept drift problem in HAR tasks using online training approaches on MCUs (Pau and Ambrose 2022; Kang et al. 2024; Craighero et al. 2023; Bian et al. 2024; Saha et al. 2024). These approaches often adapt the transfer learning paradigm to reduce resource consumption, training the neural network (composed of a feature extractor and a classifier) offline on high-performance devices and then deploying the model on MCUs. In this setup, the feature extractor is frozen, and only the classifier is trained on the MCU. For example, the recent work (Bian et al. 2024) proposed an effective solution to address EEG data distribution shifts by implementing a lightweight, efficient on-device learning engine on MCUs. This approach updated only a single linear layer on the MCU to enhance model performance for new users. The model achieved an inference time of 14.9 ms and 0.76 mJ per inference, with a 20 μ s update time and 0.83 μ J energy consumption per update during online training. However, all of these works focus primarily on reducing model performance degradation for unseen users during offline training and do not report the model's performance on old users after fine-tuning on edge devices. Fine-tuning on MCUs may degrade performance on old users, as these methods do not address the challenge of preserving old knowledge while learning new information from data streams. In this work, we propose a continual online on-device learning method to tackle this issue.

2.3 KANs in HAR

Recently, the Kolmogorov-Arnold Network (KAN) has emerged as an interesting approach to neural network design, leveraging Kolmogorov-Arnold representation theory to create potentially more efficient and interpretable models. KAN is regarded as a promising alternative to Multi-Layer Perceptrons (MLPs) (Liu et al. 2024a). With its potential, some works have explored adapting KAN to HAR tasks (Liu et al. 2024, 2024b; Herbozo Contreras et al. 2024; Chen et al. 2024; Sattari et al. 2024). For instance, Liu et al. (2024b) demonstrated an initial study in which they designed a KAN-based feature extractor for IMU-based HAR tasks. The results showed that the KAN-based feature extractor has the potential to improve HAR performance compared to a CNN-based one. Sattari et al. (2024) proposed a personalized and adaptive framework for real-time EMG-based hand motor decoding, comparing the performance of MLP-, SVM-, and KAN-based classifiers. Their results showed that the MLP-based classifier achieved the

best offline performance. Furthermore, with KAN's local plasticity property, the work (Liu et al. 2024) was the first to demonstrate KAN's potential for global incremental learning in HAR tasks. They enabled the model to continually learn multiple tasks across different datasets without significant forgetting by designing a feature redistribution layer and a KAN-based classifier. However, this approach requires incrementally adding new feature extractors for different tasks, leading to higher hardware resource consumption. Moreover, none of these works deployed KAN on MCUs or evaluated its hardware performance. Inspired by Liu et al. (2024), we propose the COOL approach, incorporating KAN as the classifier to enable HAR models to continually learn new knowledge online on MCUs. While Liu et al. (2024) demonstrated KANs for global incremental learning, their method relies on adding new feature extractors for new tasks, which increases model size over time. In contrast, our approach targets on-device learning with a fixed memory footprint, utilizing feature redistribution rather than architectural expansion. To the best of our knowledge, our proposed COOL method is the first to deploy the novel KAN-based neural network on MCUs for both inference and training, while also evaluating its latency and power consumption on edge devices.

3 Methodology

3.1 General idea

The core idea of COOL is to exploit the inherent plasticity and, crucially, the local plasticity property of KANs (Liu et al. 2024a). Unlike classical MLPs where weight updates affect the global output space, often leading to catastrophic forgetting, KANs utilize B-splines to model edge-specific functions $\phi_{i,j}(x)$ and $\Phi_i(x)$. A defining characteristic of B-splines is that their basis functions are non-zero only over a finite local interval. Consequently, the learnable parameters (control points c_i) that define the spline shape in a specific interval $[t_k, t_{k+1}]$ have zero influence on the function's output outside that interval. We leverage this property to address continual learning challenges through spatial isolation of tasks. Our method employs a Feature Redistributor to shift the distribution of input features into disjoint intervals within the KAN's input domain for each new target (task). When the model trains on a new target mapped to a new interval (e.g., $[b, c]$), backpropagation only updates the control points active in that region. The control points governing the old interval $[a, b]$, which encode the knowledge of previous targets, receive near-zero gradients and remain effectively frozen. This mechanism allows the model to extend its knowledge by growing the spline into

new regions rather than overwriting existing ones, thereby mitigating catastrophic forgetting by design.

Our method performs continuous, online on-device learning. As we can see in Fig. 1, offline learning is performed on one target and then the method continuously adapts online to new targets while retaining its performance in the previous ones.

We define continual learning in HAR as such: for the Continuous Learning Setup we have, more formally, a sequence of N Targets of new variations (either user or sensor location), $T_i \in \{T_0 \dots T_{N-1}\}$ represents each new target, indicated by each column of datasets shown in Fig. 1. Using T_0 (the offline dataset) we train both our feature extractor and classifier. Then $T_1 \dots T_{N-1}$ is the online data for the model to adapt to.

3.2 Neural network structure

At the heart of our method is its network structure, which can be seen in Fig. 2. The network starts with a 1D ResNet-based architecture feature extractor followed by a KAN classifier. The full model is first trained offline with only the data of the first target (first user or first location). After this point the feature extraction module is frozen, with only the classifier being updated online at the edge device. The neural network was implemented by PyTorch Framework.

Feature extractor: The feature extractor is designed to bring the raw signal into a useful representation. Its architecture is structured into three stages, each with a convolutional layer, two ResBlocks, and a downsampling layer, progressively increasing feature channels (32, 64, and *out_channels*), we set the *out_channels* as 64 in this work.

The output is scaled by a configured factor, incorporating dynamic adjustment based on auxiliary input. This design efficiently captures temporal features while reducing dimensionality, making it well-suited for tasks requiring deep feature learning and adaptive scaling. It incorporates ResBlocks, each consisting of two convolutional layers (kernel size 5), batch normalization, and ReLU activation. The residual connections ensure efficient learning by enabling identity mappings to bypass the convolutions. The feature extractor also includes a Downsample block, which reduces the features map resolution by half via nearest-neighbor interpolation.

The convolutional part of the feature extractor ends with a sigmoid activation, meaning we have values in the $[0, 1]$ range. Our framework allows for continuous learning by redistributing those features using our Feature Redistributor (RD) layer, which maps the features to a different space according to the task that is being evaluated. In other words, we can scale and shift features per task. This simple setup is

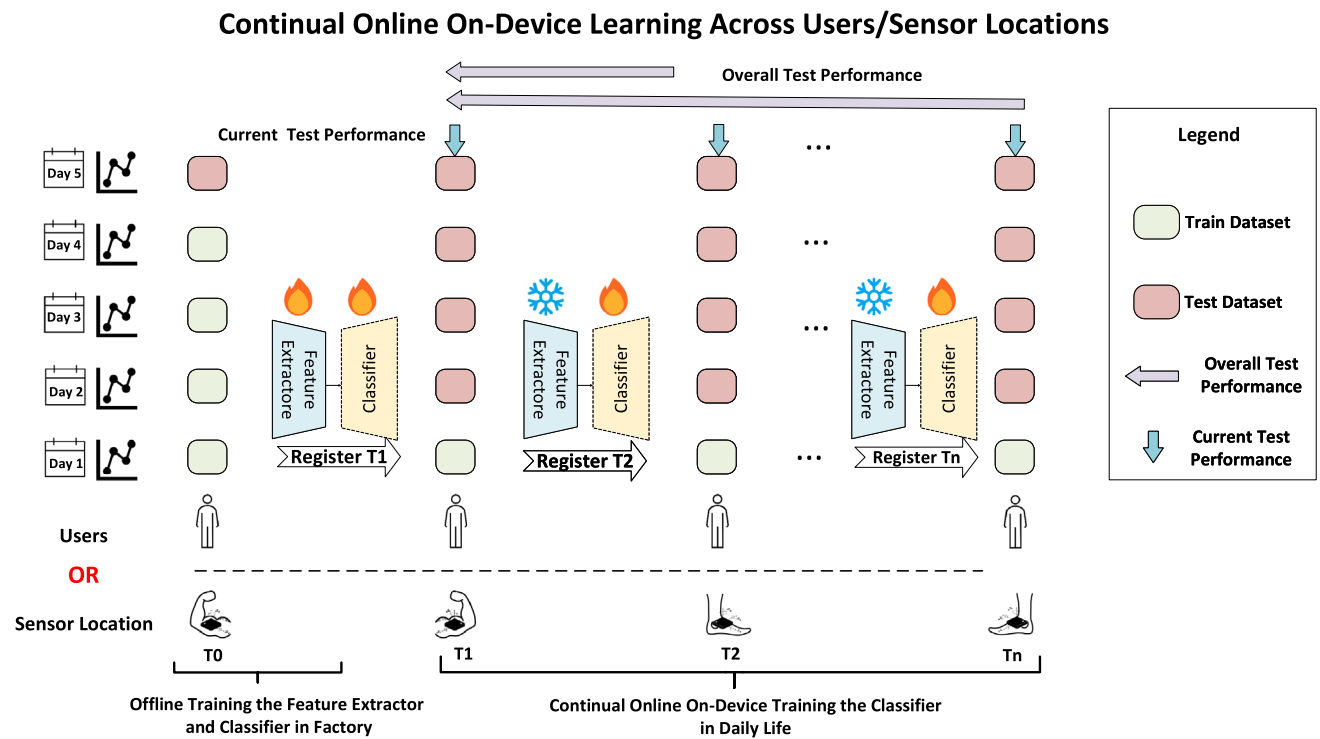


Fig. 1 Continual online on-device learning procedure across user or sensor location targets for the proposed method. Overall test performance is across previous targets while the current test performance is restricted to the last added target

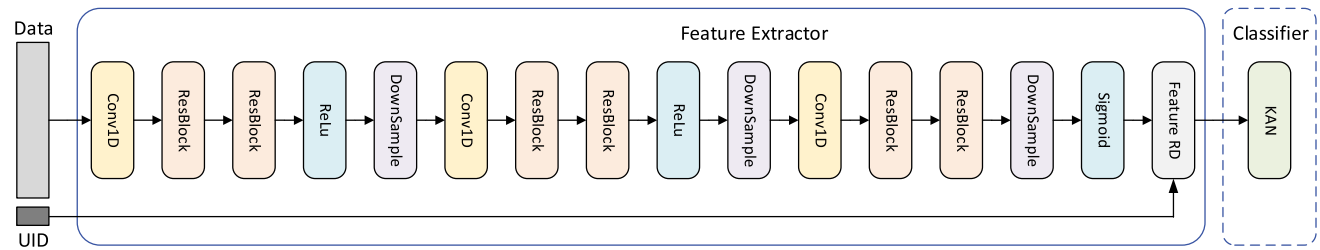


Fig. 2 The architecture of the implemented neural network. The feature extractor is offline trained and the classifier is online trained at the edge. *Feature RD* feature re-distributor layer

possible since our classifier can leverage the local plasticity property of the KANs to keep previous knowledge as well as adapt to new data.

Formally, our feature *RD* layer computes

$$RD(x, t_i) = x * \alpha + \beta * t_i \quad (1)$$

where x is the features, t_i the target index being evaluated (such as 0 for the first set, and so on) and α and β being constants representing the scaling factor and step, respectively. For α and β we used 0.08 and 0.01 since we are distributing values in the $[0, 1]$ range as the previous layer has a sigmoid activation.

KAN-based classifier: the classifier is a critical component in this work, as it must support continual online updates at edge devices. Therefore, efficiency, performance,

and memory capability must be considered simultaneously, posing a significant challenge for common classifier architectures based on dense or convolutional layers. In this work, KAN was selected as the classifier to address these challenges. KAN is built upon the B-Spline basis, a piecewise-defined polynomial that compactly represents smooth curves or functions while maintaining local control over the curve's shape.

To clearly illustrate the principle of the KAN as the classifier, we started from the B-Spline. Given a degree p , a B-Spline is defined over a knot vector $T = [t_0, t_1, \dots, t_{n+p+1}]$, which is a non-decreasing sequence of real numbers determining where and how the control points affect the curve. The basis functions $B_{i,p}(x)$ are recursively defined as follows:

Basis functions of degree 0

$$B_{i,0}(x) = \begin{cases} 1 & \text{if } t_i \leq x < t_{i+1} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

Basis functions of degree $p > 0$ For $p > 0$, the B-Spline basis functions are defined recursively as:

$$B_{i,p}(x) = \frac{x - t_i}{t_{i+p} - t_i} B_{i,p-1}(x) + \frac{t_{i+p+1} - x}{t_{i+p+1} - t_{i+1}} B_{i+1,p-1}(x) \quad (3)$$

where $B_{i,p-1}(x)$ are the basis functions of degree $p - 1$, and t_i are the knots from the knot vector. Thus, each basis function is non-zero over a small number of knot intervals, giving B-Splines local control over the curve. And a B-Spline curve can be expressed as a linear combination of basis functions and control points:

$$C(x) = \sum_i c_i B_{i,p}(x) \quad (4)$$

where c_i are the control points, and $B_{i,p}(x)$ are the B-Spline basis functions.

In the KAN architecture, B-Splines are used to model the connection-specific functions $\phi_{i,j}(x)$ and $\Phi_i(x)$. Instead of learning simple weights, the network learns the parameters control points c_i that define the B-Splines, allowing for the approximation of complex, non-linear relationships between inputs and outputs.

In this work, one-layer KAN as a classifier with M input and N output was implemented, thus, the number of the trainable parameters can be calculated along to $M * N * (K + P + 1)$, where the K is the number of the knots and P is the degree of the B-Spline. Different from the original KAN proposed in the work (Liu et al. 2024a), the bias and residual activation functions were removed to keep the local plasticity of the B-Spline. We initially configured the knot number as 30 and the degree as 1. The effect of the number of knots and degrees is present in the discussion section.

3.3 KAN trainable parameter update

Let $f: \mathbb{R}^M \rightarrow \mathbb{R}^N$ be the KAN model function mapping M -dimensional input vectors to N -dimensional output vectors. The single-layer KAN can be expressed mathematically as:

$$\phi(x) = [\phi_1(x), \phi_2(x), \dots, \phi_N(x)] \quad (5)$$

where ϕ is the transformation function for the single layer. Each ϕ_i corresponds to one output dimension, defined as:

$$\phi_i(x) = \sum_{j=1}^M \varphi_{ij}(x_j) \quad (6)$$

Here, φ_{ij} are the univariate functions on the edges, typically represented as B-Splines:

$$\varphi_{ij}(x) = \sum_{l=1}^L w_{ijl} B_l(x) \quad (7)$$

where $B_l(x)$ are the basis B-Spline functions, w_{ijl} are the learnable weights, and L is the number of basis functions used, which is related to the length of the knot vector and the degree of the B-Spline. (i : indexes the output neurons $[1, N]$; j : indexes the input dimensions $[1, M]$; l : indexes the basis functions for each edge $[1, K + P + 1]$).

The parameter updated procedure is as follows: let x : extracted feature from the feature extractor at time t , and the output: $\hat{y} = \phi(x)$, then the cross-entropy loss:

$$L(y, \hat{y}) = - \sum_{i=1}^N y_i \log(\hat{y}_i) \quad (8)$$

where y is the true label vector and $\hat{y}$ is the predicted probability vector. The backward pass to compute gradients: $\frac{\partial L}{\partial w_{ijl}}$ for all weights. Finally, we update weights using SGD:

$$g_t \leftarrow \begin{cases} \frac{\partial L}{\partial w_{ijl}}, & \text{if } t = 0 \\ \tau g_{t-1} + \frac{\partial L}{\partial w_{ijl}} & \text{if } t > 0 \end{cases} \quad (9)$$

$$w_{ijl} \leftarrow w_{ijl} - \eta g_t \quad (10)$$

where η is the learning rate and τ is the momentum. During the online training at the edge, we set the batch size as one to reduce memory usage, and we set the learning rate and momentum as 0.01 and 0.9, respectively.

3.4 Online learning engine on MCU implementation

The STM32H743 was selected as the edge device to evaluate the hardware performance of our proposed method. This high-performance microcontroller features an ARM Cortex-M7 core running at up to 480 MHz, along with 2 MB of Flash memory and 1 MB of SRAM. It also includes a single-precision floating-point unit and DSP instructions. The pretrained model was first exported as an ONNX model, which can be converted to C code using STM32Cube.AI. However, this framework only supports model inference and does not facilitate backward propagation during model training. Therefore, the on-device training procedure

was manually implemented in C. All weights of the model were stored in Flash memory, while all activations, gradient results were stored in SRAM. The weights of the classifier were read and stored in SRAM during the training phase.

4 Experiment setup

4.1 Dataset and continual learning setup

We select three public datasets in two application scenarios: concept drift caused by users and sensor location. To test the first scenario, two bio-impedance datasets were selected: the Upper-Body Fitness Activity Dataset (UBFAD) and Lower-Body Fitness Activity Dataset (LBFAD) (Liu et al. 2024). Bio-impedance signal patterns are closely related to individual body status, movements, electrode positioning, and contact surface, which leads to significant data distribution shifts among users performing similar activities. Ten subjects (4 females and 6 males), aged 21 to 32 years and weighing between 50.6 kg and 85.15 kg, performed these activities over five different days (sessions). The bio-impedance signals were collected at a sample rate of 20 Hz during the experiments. For the concept drift caused by sensor location, the Opportunity dataset (Chavarriaga et al. 2013) was used to evaluate our method. Four sensor locations were selected: right/left arm and right/left shoe. Only data from one accelerometer at each location was used. Since the purpose of this experiment was to study concept drift caused by sensor location within a user, only one user was selected for testing. The continual learning setup is shown in Fig. 1.

COOL across users experiment: This experiment was designed to evaluate the efficiency of the proposed COOL method in addressing concept shift across users, using the UBFAD and LBFAD datasets. First, data from five sessions of a single user was selected to train the entire neural network (both the feature encoder and classifier) offline, simulating a factory default setting. The data was randomly split into two parts, with four sessions used for training and one session for validation. Next, one session's data from a new user, considered as a newly registered user, was selected for online fine-tuning of the classifier at the edge. The remaining four sessions of this new user were used as the test dataset to evaluate fine-tuning performance. This procedure was repeated sequentially for the next eight users. The entire experiment was repeated ten times, with each of the ten subjects taking a turn as the factory default setting, while the remaining nine subjects served as new users in each iteration. The final evaluation was based on the average performance across all repetitions.

COOL across sensor location experiment: We further studied the performance of the proposed method for the

data distribution shift caused by the sensor location within one user. For this experiment, we selected the Opportunity dataset for evaluation (Chavarriaga et al. 2013), using the locomotion classes. While there are only four subjects in this dataset, the opportunity experiment as a focused case study on sensor placement stability rather than population-level generalization. We use as input only the acceleration and simulate a placement change in the sensor by switching between data from four locations of the first subject. In order, they are the right arm (RUA), left arm (LUA), right shoe (R-SHOE) and left shoe (L-SHOE). The experiment procedure was similar to the experiment across users, but here each task is a different sensor location. Notice that we are using only one accelerometer as input, which will of course decrease classification performance in relation to the state of the art in this dataset, but also presents a more realistic scenario where the subject is wearing only one smart device.

4.2 Evaluation metric

We developed two performance metrics based on macro F1 score of each step in the continual learning process to evaluate our approach: $\mathcal{P}_{current}$ measures the level of adaptation to the last added target, while $\mathcal{P}_{overall}$ accounts for the overall performance across all already seen targets. In all subsequent timesteps, only the classifier is fine-tuned. At each time i , our model only has access to the data of target $T_i \in \{T_0 \dots T_{N-1}\}$ mentioned in Sect. 3.1. If we denote m_i the state of our model after fine-tuning on the training set at time i , we can define

$$\mathcal{P}_{current} = \frac{1}{N} \sum_{i=1}^{N-1} perf(m_i, T_i) \quad (11)$$

$$\mathcal{P}_{overall} = \frac{1}{N} \sum_{i=1}^{N-1} \left(\frac{1}{i} \sum_{j=1}^i perf(m_i, T_j) \right) \quad (12)$$

where $perf(m, t)$ represents the macro F1 performance of our model m at the test set of t .

In summary the current metric captures the general ability of the model to learn from online data (against concept drift). The overall metric on the other hand measures the actual continual learning performance that includes the combined effect of improving on the new data and potentially forgetting information about the old (i.e. catastrophic forgetting). To specifically capture forgetting we also applied the widely used continual learning metric: forgetting measure $\mathcal{F}$ (Chaudhry et al. 2018). Forgetting is an important measure used in evaluating the performance of

continual learning models. Let $\mathcal{P}_{k,j}$ denote the performance of the model on target k after training on target j . The forgetting for target k after learning up to target t is:

$$\mathcal{F}_{k,t} = \max_{l \in \{1, \dots, t-1\}} \mathcal{P}_{k,l} - \mathcal{P}_{k,t} \quad (13)$$

where l represents the target index at which the model achieved its highest performance on target k .

The average forgetting up to target t is then calculated as:

$$\mathcal{F}_t = \frac{1}{t-1} \sum_{k=1}^{t-1} \mathcal{F}_{k,t} \quad (14)$$

These metrics help assess how well a model retains previously learned knowledge while adapting to new tasks.

4.3 Baseline and model setup

To evaluate the performance improvement by our proposed method, we defined the upper baseline and lower baseline in this work as follows:

- Upper baseline ($\mathcal{P}_{up}$): the model can access the training data from all targets during training, one-session data from all targets is used for training the model, and the remaining data.
- Lower baseline ($\mathcal{P}_{low}$): the model can only access the data from one target during training, so one-session data from one target is used for training the model, and the rest of the data from all targets is selected as test dataset.

Thus, the performance loss $\mathcal{P}_{loss}$ caused by the data distribution shift across targets is defined as (15).

$$\mathcal{P}_{loss} = \mathcal{P}_{up} - \mathcal{P}_{low} \quad (15)$$

Overall we compare our method, with its feature redistribution and KAN classifier (COOL-KAN) with fine tuning (FT) approaches that do not have a feature redistribution layer. For those FT approaches we tested with both a MLP classifier (FT-MLP) as well as KAN (FT-KAN). In order to highlight the advantages of KANs, we also tested our COOL method with a MLP classifier (COOL-MLP).

5 Results

Table 2 lists the results summary of different methods to address the data distribution shift challenges across targets, including the baseline, current performance, overall performance, and forgetting measurement. As we can see there, as well as in Fig. 4, our method has the best overall performance, meaning it can retain its knowledge on previous targets. Moreover, this comes at no or little sacrifice in current performance. Let's now go over all metrics in detail:

Baseline: the baseline results show approximately a 30% performance loss (macro F1 score) between the upper baseline (model trained using data from all targets) and the lower baseline (model trained using data from only one target) across experiments on the three datasets. This indicates a significant data distribution shift across targets. Models trained on data from a limited number of targets failed to produce accurate predictions for new targets or when the same target with a new sensor location. Therefore, online learning becomes essential after model deployment when

Table 2 Result summary of continual online learning with different methods across targets (users and sensor locations). The bolded values indicate the best results

Model	Baseline			Current performance		Overall performance		Forgetting measurement
	$\mathcal{P}_{up}$	$\mathcal{P}_{low}$	$\mathcal{P}_{loss}$	$\mathcal{P}_{current}$	Improvement	$\mathcal{P}_{overall}$	Improvement	
Continual activity recognition across users on upper-body fitness activity dataset								
FT-MLP (SotA) (Bian et al. 2024)	73.85	46.63	27.21	68.91 ± 2.30	22.28	61.16 ± 4.89	14.53	0.25 ± 0.03
FT-KAN				68.81 ± 2.55	22.18	61.70 ± 3.73	15.07	0.19 ± 0.03
COOL-MLP				55.15 ± 6.42	8.52	55.46 ± 8.66	8.83	0.09 ± 0.04
COOL-KAN (proposed)				68.75 ± 1.28	22.12	67.98 ± 1.14	21.35	0.008 ± 0.003
Continual activity recognition across users on lower-body fitness activity dataset								
FT-MLP (SotA) (Bian et al. 2024)	68.25	34.01	34.24	54.56 ± 4.69	20.55	43.99 ± 4.21	9.98	0.22 ± 0.04
FT-KAN				56.01 ± 5.05	22.00	45.13 ± 6.64	11.12	0.21 ± 0.03
COOL-MLP				43.30 ± 7.96	9.29	41.08 ± 9.23	7.07	0.13 ± 0.03
COOL-KAN (proposed)				54.15 ± 4.39	20.14	53.79 ± 4.32	19.78	0.004 ± 0.003
Continual activity recognition across sensor locations on opportunity dataset								
FT-MLP (SotA) (Bian et al. 2024)	57.03	21.10	35.93	47.87 ± 2.85	26.77	29.59 ± 7.14	8.49	0.276 ± 0.09
FT-KAN				49.29 ± 0.88	28.19	28.84 ± 3.48	7.74	0.320 ± 0.06
COOL-MLP				35.65 ± 1.46	14.55	30.51 ± 7.04	9.41	0.083 ± 0.09
COOL-KAN (proposed)				46.28 ± 8.84	25.18	46.24 ± 9.07	25.14	0.001 ± 0.01

new targets are introduced in practical applications. Additionally, we selected only one session as the training dataset, with the remaining four sessions used for testing. As a result, the upper baseline performance is not as high as in existing studies, which typically leave only one session out for testing.

Current performance: the current performance is used to only evaluate the model performance on the current target after being updated, which is often reported in works based on fine tuning (Bian et al. 2024). It can be observed that the best current results were achieved by the fine tuning (FT) based method among the three datasets, the combination of the FT and KAN realized better current performance in the LBFAD and Opportunity datasets, which is around 2% higher than the FT-MLP model. In the UBFAD datasets, the current performance of the FT-MLP is around 0.1% better than the FT-KAN model. The COOL-MLP models with the feature redistribution layer demonstrated the worst current performance among all three datasets as shown in Fig. 4, resulting from the designed feature redistribution layer. However, the COOL-KAN models achieved close results to the best current performance. Thus, the implemented feature redistribution layer has little effect on the performance of the COOL-KAN model. Overall, the activity recognition results on the current targets have been significantly improved after the model update among all four models.

Overall performance: we also reported the overall performance, which evaluates the model's performance on all registered targets. This is measured by the average macro F1 score across all targets after the model is updated with data from the last target. As shown in Fig. 4 and Table 2, our proposed COOL-KAN model demonstrated significantly better overall performance across targets compared to the other models. For example, in the continual learning experiment across sensor locations, the average overall performance improved by 25.14% using the COOL-KAN model, compared to the lower baseline without model updates from new data. In contrast, the other models showed less than a 10% improvement in average overall performance. Furthermore, while FT-MLP and FT-KAN achieved better improvements in current performance, their average overall performance increased by only around 8%. Similar trends were observed in the other experiments, highlighting the critical role the feature redistribution layer plays in enhancing the overall performance of continual online learning models.

Forgetting measurement: the forgetting measurement was used to evaluate the model's memory retention after being updated with new data – the lower the forgetting score, the better the model's memory ability. Our proposed COOL-KAN model achieved the lowest forgetting score, below 0.01 in all three experiments, indicating superior memory retention. The COOL-MLP model, with the Feature RD

layer, also demonstrated a notably lower forgetting score. Figure 3 illustrates the evolution of user performance as the current subject changes in the UBFAD dataset. It shows that the performance on previous users degrades when using FT-MLP and FT-KAN models, whereas our approach mitigates this performance loss after updating the model with data from new targets.

Overall, the experimental results demonstrate that the proposed COOL-KAN model not only significantly improves the activity recognition performance for current targets through fine-tuning (with improvements similar to those of FT-KAN and FT-MLP) but also achieves excellent overall performance across all targets. This overall performance is close to the current performance of each target, indicating that the COOL-KAN model effectively addresses the forgetting issue after updates. Moreover, the consistent results across all three datasets in addressing data distribution shifts across targets underscore the robustness of the COOL-KAN model, as shown in Fig. 4 and Fig. 5.

Hardware implementation results: the on-edge performance of our proposed model was demonstrated by deploying it on the STM32H743 microcontroller, where it was able to perform both inference and classifier training. Hardware performance metrics, such as power consumption and latency, were reported. Figure 6 illustrates the power consumption measurement setup. The STM32 Power Shield board (X-NUCLEO-LPM01A) was used to dynamically measure power consumption during inference and training. The system clock frequency of the microcontroller was set to 480 MHz. Latency was measured using the microcontroller's high-precision 16-bit counter. Table 3 summarizes the hardware implementation results for our proposed COOL-KAN model. Since the microcontroller was connected to several peripheral modules, only dynamic power consumption was measured. Model inference and training were triggered by input commands from a laptop via the UART interface. The microcontroller's standby state, where it waits for commands, was defined as the idle status. The results showed that the dynamic power consumption during standby was approximately 18 mW, while inference consumed around 22 mW. On-device training required only about 1 mW more than inference. Each inference operation consumed approximately 2.7 mJ of energy, and each training cycle consumed around 2.8 mJ. The inference time on the STM32H743 was around 45 ms, with an additional 1.7 ms required to update the classifier's weights.

Fig. 3 The results of the learning process of different methods (upper-body fitness activity dataset)

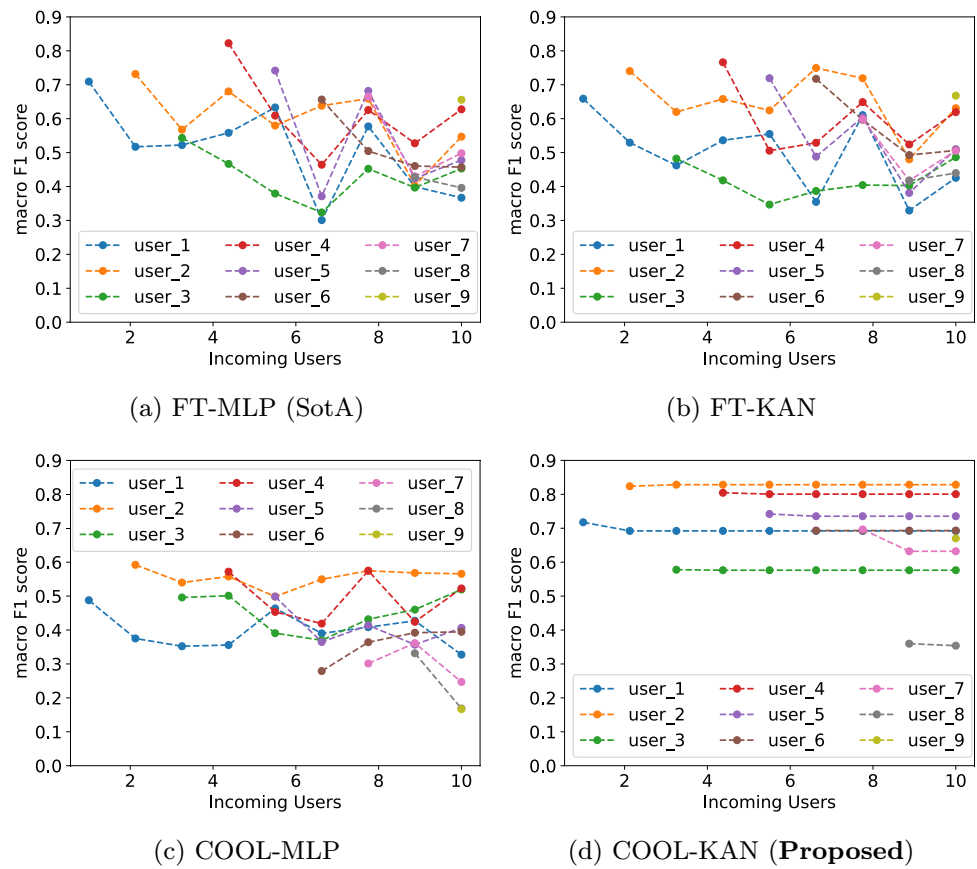


Fig. 4 Current and overall performance

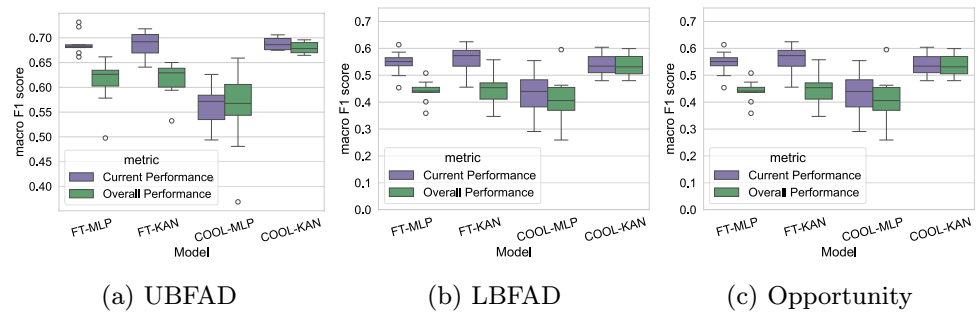
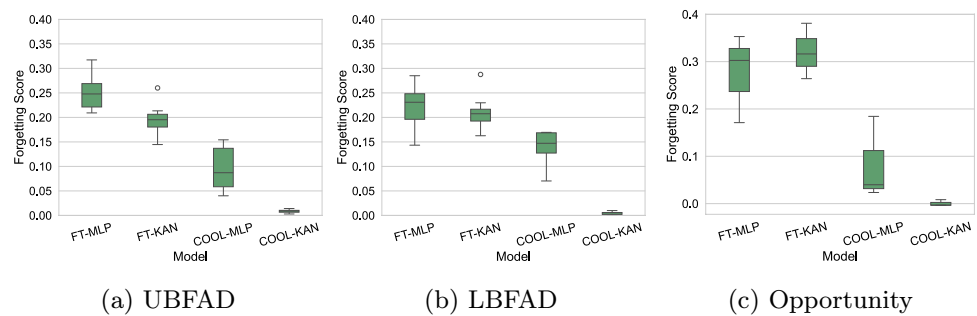


Fig. 5 Forgetting measurement performance (the small one is preferable)



6 Discussion

Since the number of knots and degree of the B-Spline are two of the important hyperparameters, that have a potential effect on the performance of our proposed COOL-KAN

model, a further experiment to study their influence on the COOL-KAN model was conducted. Five different knot numbers from 10 to 50 with a step size of 10 and three degrees from 1 to 3 were tested, and the test results are as shown in Fig. 7, Fig. 6 and Fig. 9.

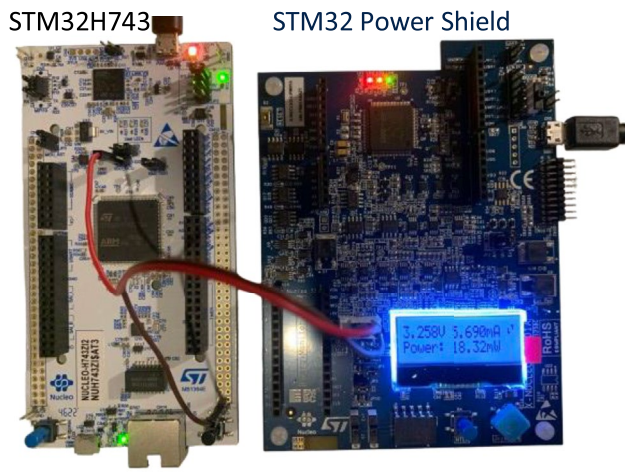


Fig. 6 Power consumption measurement setup

Table 3 Hardware Implementation Results (STM32H743 @480MHz)

Phase	Metric	Upper-body	Lower-body	Opportunity
Standby	Power ¹ (mW)	18.33		
Inference	Power (mW)	22.36	22.31	22.31
	Latency (ms)	44.85	44.79	44.72
	Flash (KiB)	946.89	953.74	954.38
	SRAM (KiB)	31.75	32.51	32.51
	MACC (K)	3.300	3.296	3.303
	cycles/MACC	6.52	6.52	6.50
	Energy(mJ)/Inf	2.72	2.62	2.62
Training	Power (mW)	23.43	23.42	23.42
	Latency ²	1.73	1.72	1.72
	Energy(mJ)/Train	2.85	2.84	2.84

¹All the power consumption is the dynamic power consumption. ² This latency only includes the time of the classifier weights update per one sample, the time cost of the feature extraction is not included. The Latency per training step is around 46 ms (including forward and backward pass)

6.1 The effect of knot number

It can be observed that the knot number has significant effect on the overall performance and forgetting measurement, there is an obvious overall performance degradation when the grid number is less than 30, such result can be found among all three experiments, as Fig. 7b, Fig. 8b and Fig. 9b show. The knot number also has similar effect on the forgetting measurement, whereas it has less effect on the current performance. And the best current performance was not achieved by the model with the largest knot number, as shown in Fig. 9a. In addition, the knot number is also an important factor in model size computation, determining whether the model can be deployed on the edge devices, thus, an optimized knot number configuration is very important. We hypothesize that the spline learned by the KAN at each interval is controlled by points inside and outside it, with those outside having small enough values (and influence) so that those can then correctly adapt to the next intervals without deforming the previous structure.

6.2 The effect of B-spline degree

The B-Splines are smooth up to a degree determined by the degree of the spline, which is critical for the continual function fitting accuracy. However, in our classification tasks, it can be found that the higher degree number can not increase the classification results, it could even reduction of current and overall performance, as Fig. 8. Besides, the results also showed that it requires more knots to achieve the same forget measurement scores when the degree is increased. What's more, the higher degree requires more computation resource and time when compute the B-Spline basis in KAN model. Thus, the results from our study suggested that the

Fig. 7 The effect of the number of knots and degree on the results. The green color is preferable. (UBFAD)

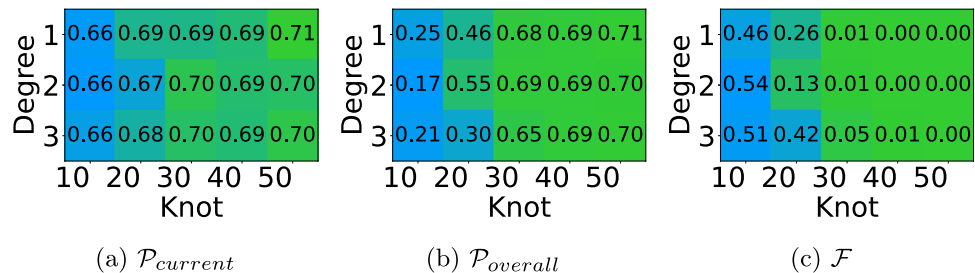


Fig. 8 The effect of the number of knots and degree on the results. The green color is preferable. (LBFAD)

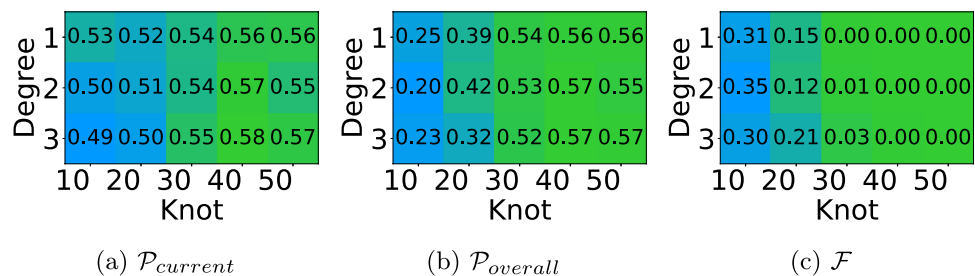
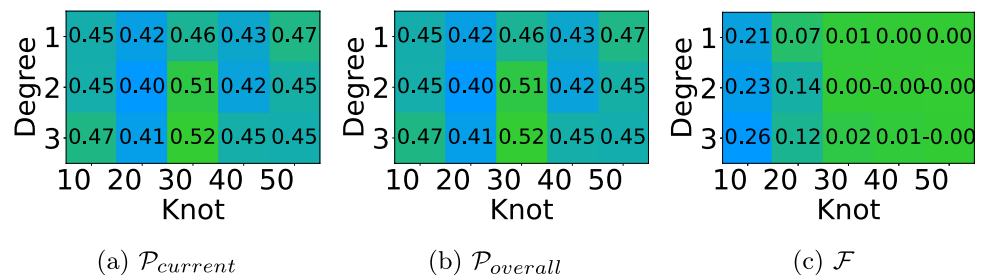


Fig. 9 The effect of the number of knots and degree on the results. The green color is preferable. (Opportunity)



degree should be kept as small as possible to improve the efficiency and continual learning performance in the classification task.

6.3 Baselines and hardware constraints

A significant consideration in this work is the strict hardware constraints imposed by the microcontroller environment (STM32H743 with 1MB SRAM). While established continual learning strategies such as Experience Replay (ER) or EWC are effective in server-side scenarios, they are often infeasible for on-device training. ER requires maintaining a data buffer in the limited SRAM, where storing even a small fraction of historical high-dimensional sensor data competes with the memory required for gradient computation and activations during backpropagation. Similarly, regularization-based methods like EWC require storing an importance matrix (Fisher Information Matrix) of the same size as the model parameters, effectively doubling the memory footprint of the classifier. In contrast, our COOL framework requires negligible additional memory overhead beyond the standard training requirements. Consequently, we limited our baseline comparisons to Fine-Tuning, as it represents the most viable state-of-the-art approach for performing adaptation on constrained edge devices without requiring external memory expansion.

6.4 Limitations

Although our proposed COOL-KAN model demonstrated superior results in terms of overall performance and forgetting measurement compared to other models, several limitations remain. First, there is no efficient concept drift detector to indicate when the model should be updated. In our current implementation, the model is trained immediately when a new target is registered, which could lead to resource waste if the data distribution between new and old targets is similar. Moreover, the system requires the target ID, which in future work could be automatically inferred. Another area that needs further study is the evolution of performance when we significantly increase the number of tasks. This could be a limiting factor related to the KANs generalisation capacity as we do not increase the number

of parameters. This includes exploring the limitations of KANs in continuous learning.

Additionally, a few-shot training method was not utilized in this work. Instead, a complete session's data was used for fine-tuning the model, leaving considerable room for improvement in terms of efficiency.

Most importantly, our method relies on the local plasticity of the KAN classifier, which does not lead to any increase in parameters as more targets are added within the limitation of the determined number of knots. Unintuitively, incrementing the targets within our benchmark datasets also does not lead to computation or memory increase. Unlike conventional neural networks like MLP, KAN requires an appropriately fixed knot capacity to facilitate decent learning results, as shown in Sect. 6.1. In our case, a small knot number (30) was sufficient, but bigger sizes could make the model size bigger than an equivalent MLP. However, the upper bound of new targets permitted by the number of knots has not been fully explored in the current work, because the number of knots optimal for this scenario appears to be ample for the amount of targets in the datasets. Nonetheless, from the comparison between COOL-KAN and COOL-MLP (Table 2) we can see superior continual learning properties in both memory retention and new concept adaptation for COOL-KAN.

Besides, our Feature Redistributor relies on an explicit target index t_i to map input features to disjoint B-Spline intervals. This dependency currently restricts the method to “task-incremental” scenarios, where the system is externally notified of a context change (e.g., a user manually selecting a “New Position” profile or a “New User” calibration mode). This contrasts with fully unsupervised “in-the-wild” settings where boundaries between contexts are unknown. Future work will aim to remove this reliance by integrating an unsupervised concept drift detector. By monitoring statistical shifts in the input distribution or drops in prediction confidence, the system could automatically infer the onset of a new task and increment the redistribution index t_i without human intervention.

7 Conclusion

In conclusion, we introduced COOL, a continual online on-device learning method enhanced by Kolmogorov-Arnold networks (KANs). COOL enables neural networks to be continually trained on edge devices using data from new targets without forgetting knowledge from previous ones. Our experimental results show that COOL significantly improves overall performance, with average gains of 21.35% and 19.78% in upper-body and lower-body activity recognition, and a 25.14% improvement on the IMU dataset, all while maintaining a forgetting score of less than 0.01. Additionally, COOL demonstrated efficient hardware performance, with inference and training times of 44.30 ms and 46.03 ms, respectively, on the STM32H743 microcontroller. This work confirmed the potential of the proposed COOL model to address data distribution shift challenges in HAR tasks, enabling effective and efficient deployment on edge devices.

Acknowledgements This paper is supported by the EU Horizon research and innovation program under grant agreement No 101070408, project SustainML, and by the Carl-Zeiss Stiftung under the Sustainable Embedded AI project (P2021-02-009).

Author contributions Conceptualization, M.L. and V.R.; data processing, M.L. and S.B. and P.K.; investigation, M.L. and S.B.; methodology, M.L. and B.Z.; resources, P.L.; software, M.L.; supervision, P.L.; validation, M.L., B.Z. and V.R.; visualization, M.L.; writing-original draft, M.L.; writing, M.L.; review and editing, B.Z., V.R., D.G., S.B. and P.L. All authors have read and agreed to the published version of the manuscript.

Funding Open Access funding enabled and organized by Projekt DEAL.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Adaimi, R., Thomaz, E.: Lifelong adaptive machine learning for sensor-based human activity recognition using prototypical networks. *Sensors* **22**(18), 6881 (2022)
- Ariza-Colpas, P.P., Vicario, E., Oviedo-Carrascal, A.I., Butt Aziz, S., Pineres-Melo, M.A., Quintero-Linero, A., Patara, F.: Human activity recognition data analysis: history, evolutions, and new trends. *Sensors* **22**(9), 3401 (2022)
- Bian, S., Lukowicz, P.: Capacitive sensing based on-board hand gesture recognition with tinyml. In: Adjunct Proceedings of the 2021 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2021 ACM International Symposium on Wearable Computers, pp. 4–5 (2021)
- Bian, S., Kang, P., Moosmann, J., Liu, M., Bonazzi, P., Rosipal, R., Magno, M.: On-device learning of eegnet-based network for wearable motor imagery brain-computer interface. *arXiv preprint arXiv:2409.00083* (2024)
- Chaudhry, A., Dokania, P.K., Ajanthan, T., Torr, P.H.: Riemannian walk for incremental learning: Understanding forgetting and intransigence. In: Proceedings of the European Conference on Computer Vision (ECCV), pp. 532–547 (2018)
- Chavarriaga, R., Sagha, H., Calatroni, A., Digumarti, S.T., Tröster, G., Millán, J.D.R., Roggen, D.: The opportunity challenge: A benchmark database for on-body sensor-based activity recognition. *Pattern Recognition Letters* **34**(15), 2033–2042 (2013)
- Chen, Z., Liu, B.: Continual learning and catastrophic forgetting. In: Lifelong machine learning, pp. 55–75. Springer, Cham (2018)
- Chen, Z., Li, X., Huang, Q., Geng, Q., Yang, T., Han, S.: Kan-hyperpointnet for point cloud sequence-based 3d human action recognition. *arXiv preprint arXiv:2409.09444* (2024)
- Craigheo, M., Quarantiello, D., Rossi, B., Carrera, D., Fragneto, P., Boracchi, G.: On-device personalization for human activity recognition on stm32. In: IEEE embedded systems letters. IEEE, Geneva (2023)
- David, R., Duke, J., Jain, A., Janapa Reddi, V., Jeffries, N., Li, J., Kreeger, N., Nappier, I., Natraj, M., Wang, T., et al.: Tensorflow lite micro: embedded machine learning for tinyml systems. *Proc. Mach. Learn. Syst.* **3**, 800–811 (2021)
- Herbozo Contreras, L.F., Cui, J., Yu, L., Huang, Z., Nikpour, A., Kavehei, O.: Kan-eeg: Towards replacing backbone-mlp for an effective seizure detection system. *medRxiv*, 2024–06 (2024)
- Host, K., Ivašić-Kos, M.: An overview of human action recognition in sports based on computer vision. *Heliyon* **8**(6), e09633 (2022)
- Ingolfsson, T.M., Chakraborty, U., Wang, X., Beniczky, S., Ducouret, P., Benatti, S., Ryvlin, P., Cossetini, A., Benini, L.: Epidenet: An energy-efficient approach to seizure detection for embedded systems. In: 2023 IEEE Biomedical Circuits and Systems Conference (BioCAS), pp. 1–5 (2023). IEEE
- Ingolfsson, T.M., Wang, X., Chakraborty, U., Benatti, S., Bernini, A., Ducouret, P., Ryvlin, P., Beniczky, S., Benini, L., Cossetini, A.: Brainfusenet: Enhancing wearable seizure detection through eeg-ppg-accelerometer sensor fusion and efficient edge deployment. *IEEE Transactions on Biomedical Circuits and Systems* (2024). <https://doi.org/10.1109/TBCAS.2024.3395534>
- Jin, X., Sadhu, A., Du, J., Ren, X.: Gradient-based editing of memory examples for online task-free continual learning. *Adv. Neural. Inf. Process. Syst.* **34**, 29193–29205 (2021)
- Kang, P., Moosmann, J., Bian, S., Magno, M.: On-device training empowered transfer learning for human activity recognition. *arXiv preprint arXiv:2407.03644* (2024)
- Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., Zhang, G.: Learning under concept drift: a review. *IEEE Trans. Knowl. Data Eng.* **31**(12), 2346–2363 (2018)

- Lee, S., Ha, J., Zhang, D., Kim, G.: A neural dirichlet process mixture model for task-free continual learning. *arXiv preprint arXiv:2001.00689* (2020)
- Liu, X., Masana, M., Herranz, L., Weijer, J., Lopez, A.M., Bagdanov, A.D.: Rotate your networks: Better weight consolidation and less catastrophic forgetting. In: 2018 24th International Conference on Pattern Recognition (ICPR), pp. 2262–2268 (2018). IEEE
- Liu, M., Bian, S., Zhou, B., Lukowicz, P.: ikan: Global incremental learning with kan for human activity recognition across heterogeneous datasets. *arXiv preprint arXiv:2406.01646* (2024)
- Liu, M., Geißler, D., Nshimiyimana, D., Bian, S., Zhou, B., Lukowicz, P.: Initial investigation of kolmogorov-arnold networks (kans) as feature extractors for imu based human activity recognition. *arXiv preprint arXiv:2406.11914* (2024)
- Liu, M., Rey, V.F., Zhang, Y., Ray, L.S.S., Zhou, B., Lukowicz, P.: imove: Exploring bio-impedance sensing for fitness activity recognition. In: 2024 IEEE International Conference on Pervasive Computing and Communications (PerCom), pp. 194–205 (2024). IEEE
- Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljačić, M., Hou, T.Y., Tegmark, M.: Kan: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756* (2024)
- Leite, C.F.S., Xiao, Y.: Resource-efficient continual learning for sensor-based human activity recognition. *ACM Trans. Embedded Comput. Syst.* **21**(6), 1–24 (2022)
- Liang, T., Glossner, J., Wang, L., Shi, S., Zhang, X.: Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing* **461**, 370–403 (2021)
- Nagel, M., Fournarakis, M., Amjad, R.A., Bondarenko, Y., Van Baalen, M., Blankevoort, T.: A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295* (2021)
- Nwakanma, C.I., Islam, F.B., Maharani, M.P., Lee, J.-M., Kim, D.-S.: Detection and classification of human activity for emergency response in smart factory shop floor. *Applied Sciences* **11**(8), 3662 (2021)
- Pau, D., Ambrose, P.K.: Automated neural and on-device learning for micro controllers. In: 2022 IEEE 21st Mediterranean electro-technical conference (MELECON), pp. 758–763. IEEE, Geneva (2022)
- Saha, B., Samanta, R., Roy, R.B., Chakraborty, C., Ghosh, S.K.: Personalized human activity recognition: real-time on-device training and inference. In: IEEE consumer electronics magazine. IEEE, Geneva (2024)
- Sattari, P., Ravanshid, D., Nasiri, R.: Designing for practicality: A personalized and adaptive framework for real-time emg-based hand motor decoding. *Authorea Preprints* (2024)
- Schiemer, M., Fang, L., Dobson, S., Ye, J.: Online continual learning for human activity recognition. *Pervasive and Mobile Computing* **93**, 101817 (2023)
- Shanahan, M., Kaplanis, C., Mitrović, J.: Encoders and ensembles for task-free continual learning. *arXiv preprint arXiv:2105.13327* (2021)
- Shin, H., Lee, J.K., Kim, J., Kim, J.: Continual learning with deep generative replay. *Advances in neural information processing systems* **30** (2017)
- Siirtola, P., Rönning, J.: Context-aware incremental learning-based method for personalized human activity recognition. *J. Ambient. Intell. Humaniz. Comput.* **12**(12), 10499–10513 (2021)
- Tang, X., Yao, L., Wang, S., Sheng, Q.Z., Gu, T., Xiong, N.: Balancing continual learning and fine-tuning for human activity recognition. *arXiv preprint arXiv:2401.02255* (2024)
- Ven, G.M., Tuytelaars, T., Tolias, A.S.: Three types of incremental learning. *Nature Machine Intelligence* **4**(12), 1185–1197 (2022)
- Vakili, M., Rezaei, M.: Incremental learning techniques for online human activity recognition. *arXiv preprint arXiv:2109.09435* (2021)
- Vrigkas, M., Nikou, C., Kakadiaris, I.A.: A review of human activity recognition methods. *Frontiers in Robotics and AI* **2**, 28 (2015)
- Wang, Y., Cang, S., Yu, H.: A survey on wearable sensor modality centred human activity recognition in health care. *Expert Systems with Applications* **137**, 167–190 (2019)
- Yu, H., Chen, Z., Zhang, X., Chen, X., Zhuang, F., Xiong, H., Cheng, X.: Fedhar: Semi-supervised online learning for personalized federated human activity recognition. *IEEE Trans. Mob. Comput.* **22**(6), 3318–3332 (2021)

Mengxi Liu is currently a Ph.D. Researcher at RPTU and DFKI, Germany. He is working towards the intersection of novel sensing hardware and advanced machine learning architectures for real-world activity recognition.

Sizhen Bian received the B.S. degree from the Northwestern Polytechnical University, China, in 2013, and the M.S. degree and Ph.D. degree in computer science from the Technische Universität Kaiserslautern, Kaiserslautern, Germany, in 2016 and 2022, respectively. From 2022 to 2024, he was a Senior Researcher with ETH Zürich, Zürich, Switzerland. Since 2025, he has been a Senior Researcher at DFKI and RPTU. His research interests include spiking neural networks, pervasive computing, and efficient edge computing.

Vitor Fortes Rey has been a Senior Researcher at the German Research Center for Artificial Intelligence (DFKI), Germany, since 2019. He received his Ph.D. in Computer Science from RPTU Kaiserslautern–Landau, Germany, in 2024, and his B.S. and M.S. degrees from the Universidade Federal do Rio Grande do Sul, Porto Alegre, Brazil, in 2013 and 2016, respectively. His research interests include machine learning, multimodal human activity recognition, and knowledge engineering.

Daniel Geißler is a Researcher and PhD candidate at DFKI, Kaiserslautern, Germany. His research focuses on efficient and sustainable Machine Learning, optimizing the pipeline from input sensing modalities till sophisticated model predictions.

Pixi Kang was born in Shandong, China, in 1999. He received the B.E. degree in microelectronics science and engineering from Tsinghua University, Beijing, China, in 2021. He is currently pursuing the M.S. degree in integrated circuits engineering with Tsinghua Shenzhen International Graduate School, School of Integrated Circuits, Tsinghua University. His research interests include radar signal processing, machine learning, and lightweight algorithms for IoT devices.

Bo Zhou is a Senior Researcher at the German Research Center for Artificial Intelligence (DFKI GmbH). His main research interests are sensor-based perception through hardware-software co-designed systems and sensor signal analysis. He received his Ph.D. in Computer Science at RPTU Kaiserslautern–Landau, and his M.S. degrees in Electronics from the University of Southampton, RPTU Kaiserslautern–Landau and NTNU

Paul Lukowicz is both Scientific Director at the German Research Center for Artificial Intelligence (DFKI GmbH) and Professor of Computer Science at RPTU Kaiserslautern–Landau in Germany since 2012 where he heads the Embedded Intelligence group. His research focuses on context-aware ubiquitous and wearable systems including sensing, pattern recognition, system architectures, models of largescale self-organized systems, and applications in areas ranging from healthcare through industry 4.0 to smart cities.